PHASE 15 — MULTI-SOURCE DISCOVERY ENGINE & ENTERPRISE SCANNING FABRIC

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 15 of 22

Status: Enterprise Discovery, Scanner Orchestration & Collection Architecture

---

1. Purpose of This Phase

The previous phases established the intelligence layer:

```
DISCOVERY
   ↓
CBOM
   ↓
KNOWLEDGE GRAPH
   ↓
QUANTUM RISK
   ↓
PQC RECOMMENDATION
   ↓
AI INTELLIGENCE
```

But none of that works without one fundamental capability:

ECDAT must actually find the cryptographic artifacts.

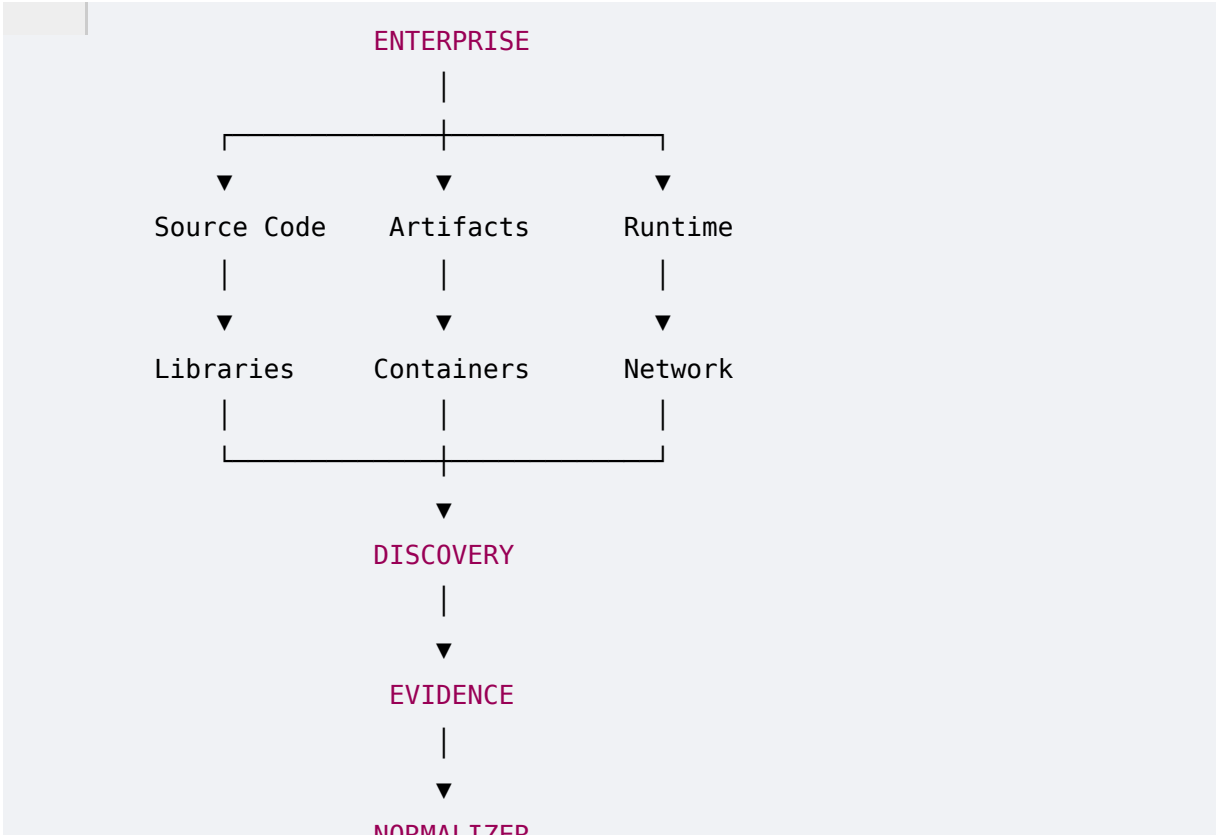
The SIH problem statement specifically calls for scanning:

- Source code repositories
- Binaries
- Libraries
- Container images

and identifying cryptographic artifacts across applications and infrastructure.

A serious enterprise implementation should go beyond those minimum requirements.

The discovery architecture should be:



---

2. The Discovery Problem

Enterprise cryptography is not located in one place.

It can exist in:

```
Source code
Dependency files
Compiled binaries
Container images
Operating systems
Certificates
TLS endpoints
HSMs
Cloud KMS
Configuration
CI/CD
Kubernetes
Mobile applications
Firmware
Embedded devices
```

Therefore:

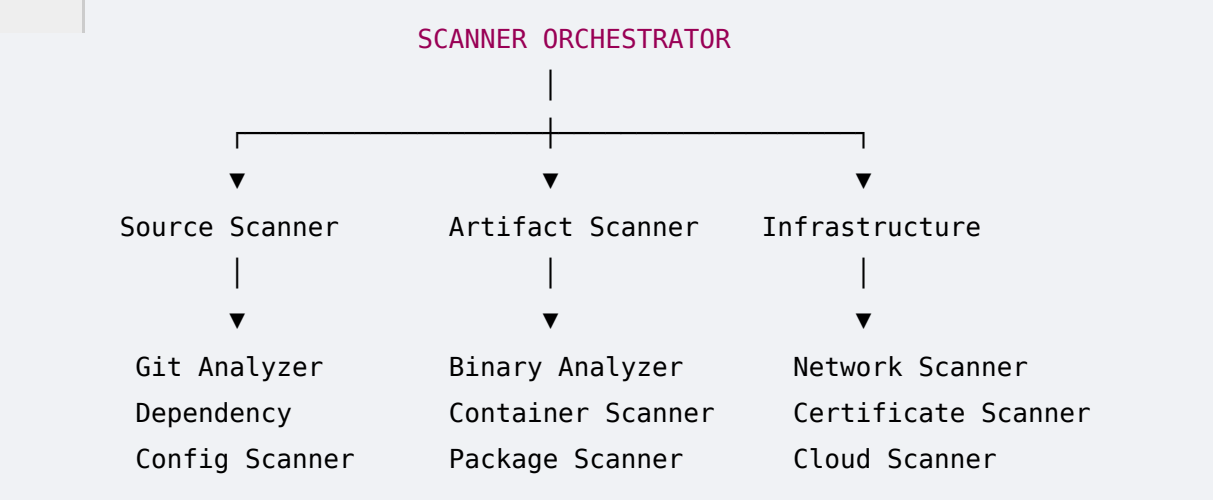
No single scanner can provide complete cryptographic visibility.

ECDAT should use a **scanner fabric**.

---

3. Scanner Fabric

The architecture should consist of specialized scanners.



Each scanner has one job.

---

4. Why Specialized Scanners?

A source-code scanner understands:

```
Functions
Imports
APIs
```

A binary scanner understands:

```
Symbols
Sections
Instructions
Libraries
```

A certificate scanner understands:

```
X.509
ASN.1
```

```
OIDs
Validity
```

Trying to make one scanner handle all of these creates unnecessary complexity.

---

5. Scanner Orchestrator

ECDAT needs a central component:

Scanner Orchestrator.

Its responsibilities:

```
Create scan jobs
Assign scanners
Track progress
Retry failures
Collect results
Enforce permissions
Track coverage
Normalize results
```

---

6. Scan Job

Every scan should become a job.

Example:

```
{
  "scan_id": "scan-2026-001",
  "target": "repository://payment-api",
  "scanner": "source-code",
  "priority": "high",
  "status": "queued"
}
```

---

7. Scan Lifecycle

```
QUEUED
  ↓
STARTING
  ↓
RUNNING
```

```
      ↓
PROCESSING
      ↓
NORMALIZING
      ↓
COMPLETED
```

Failure:

```
RUNNING
   ↓
FAILED
   ↓
RETRY
   ↓
RUNNING
```

---

8. Scan Types

ECDAT should support:

```
FULL
INCREMENTAL
TARGETED
SCHEDULED
EVENT-TRIGGERED
AD-HOC
CONTINUOUS
```

---

9. Full Scan

A full scan analyzes everything.

Useful for:

```
Initial onboarding
Major migration
Baseline inventory
Audit
```

---

10. Incremental Scan

Only changed artifacts are analyzed.

Example:

```
Commit
 ↓
Changed Files
 ↓
Affected Dependencies
 ↓
Targeted Scan
```

This is much faster.

---

11. Targeted Scan

Example:

> Scan only cryptographic configuration in production.

Or:

> Scan this container image.

---

12. Scheduled Scan

Organizations may configure:

```
Daily
Weekly
Monthly
```

depending on environment.

---

13. Event-Triggered Scan

Example:

```
Git Push
 ↓
CI/CD
 ↓
ECDAT Scan
```

Or:

```
New Container Image
 ↓
Registry Event
 ↓
ECDAT Scan
```

---

14. Continuous Discovery

For large organizations:

```
New Asset
   ↓
Automatically discovered
   ↓
Automatically scanned
   ↓
CBOM updated
```

This creates a continuously evolving inventory.

---

15. Source-Code Scanner

The first major scanner:

Source Code Discovery Engine.

It should support common ecosystems.

Potential languages:

```
C
C++
Java
Go
Rust
Python
JavaScript
TypeScript
C#
Kotlin
Swift
PHP
Ruby
```

The initial prototype does not need all of these.

A strategically selected subset is enough for an SIH demonstration.

---

16. What the Source Scanner Looks For

Not just algorithm names.

It should detect:

```
Crypto APIs
Crypto libraries
Key operations
Certificate operations
Hashing
Signing
Encryption
Decryption
Randomness
TLS
SSH
JWT
Password hashing
Custom crypto
```

---

17. Imports

Example:

```
import cryptography
```

or:

```
from Crypto.Cipher import AES
```

These create evidence.

But import alone does not prove actual usage.

---

18. API Calls

Example:

```
AES.new(...)
```

is stronger evidence.

ECDAT should classify:

```
Library
API
Operation
Algorithm
Parameters
```

---

19. Function Calls

Example:

```
EVP_EncryptInit_ex()
```

should map to:

```
OpenSSL
  ↓
Encryption API
```

Further context determines the algorithm.

---

20. Constants

Look for:

```
AES-256
RSA-2048
P-256
SHA-256
TLS_AES_256_GCM_SHA384
```

Constants can provide strong clues.

---

21. Configuration

Crypto may be configured externally:

```
algorithm: RSA
key_size: 2048
```

Therefore configuration scanners should be integrated.

---

22. Dependency Scanner

Dependency files include:

```
package.json
package-lock.json
pom.xml
build.gradle
requirements.txt
Cargo.toml
go.mod
composer.json
Gemfile
```

The scanner extracts:

```
Package
Version
Source
Dependency Graph
```

---

23. Why Dependencies Matter

Suppose the source code doesn't explicitly import OpenSSL.

But:

```
Application
 ↓
Library A
 ↓
OpenSSL
```

ECDAT needs to know that cryptographic capability exists.

---

24. Dependency vs Actual Usage

This distinction is crucial.

```
OpenSSL installed
```

does not necessarily mean:

```
Application actively uses ECDSA
```

Therefore:

```
Dependency Evidence
```

should have lower confidence than:

```
Direct API Evidence
```

unless corroborated.

---

25. Binary Scanner

Source code is often unavailable.

Enterprise applications may only provide:

```
Executable
Shared Library
Firmware
JAR
APK
Container
```

Therefore ECDAT needs binary analysis.

---

26. Binary Formats

Potential formats:

```
ELF
PE
Mach-O
JAR/WAR
DEX
WASM
Firmware
```

---

27. ELF Analysis

For Linux binaries:

```
Sections
Symbols
Imports
Exports
Dynamic Dependencies
Strings
```

can reveal cryptographic components.

---

28. PE Analysis

For Windows:

```
Imports
Exports
DLL dependencies
Strings
Metadata
Sections
```

can reveal:

```
BCrypt
CNG
OpenSSL
CryptoAPI
```

and other cryptographic dependencies.

---

29. Mach-O Analysis

For macOS/iOS ecosystems:

```
Frameworks
Symbols
Linked Libraries
Sections
Strings
```

can reveal cryptographic dependencies.

---

30. JAR/WAR Analysis

Java applications can be inspected for:

```
Java Cryptography Architecture
Bouncy Castle
Conscrypt
OpenSSL bindings
TLS configuration
```

---

31. Mobile Applications

For Android:

```
APK
DEX
Native Libraries
Manifest
Resources
```

may reveal cryptographic usage.

For iOS:

```
IPA
Frameworks
Native binaries
```

may be analyzed where authorized.

---

32. String Analysis

Binaries often contain:

```
RSA
AES
ECDSA
SHA256
EVP
TLS
```

String matches are useful but noisy.

Therefore:

```
String Evidence
```

should have lower confidence than:

```
Verified API Call
```

---

33. Symbol Analysis

Symbols can be powerful.

Example:

```
EVP_PKEY_sign
EVP_sha256
RSA_public_encrypt
```

These provide stronger evidence of cryptographic behavior.

---

34. Disassembly

For difficult binaries:

```
Binary
 ↓
Disassembly
 ↓
Instruction Patterns
 ↓
Crypto Identification
```

This is computationally expensive and should be reserved for cases where cheaper techniques fail.

---

35. Container Scanner

Modern enterprises deploy applications through containers.

ECDAT should scan:

```
Docker images
OCI images
Container layers
```

---

36. Container Layer Analysis

A container may contain:

```
Base image
+
System libraries
+
Application
+
Crypto libraries
+
Certificates
+
Configuration
```

ECDAT should analyze each layer.

---

37. Base Image

Example:

```
ubuntu:24.04
```

The scanner identifies:

```
OS packages
OpenSSL
GnuTLS
Crypto libraries
```

---

38. Application Layer

The container may include:

```
Python package
Java JAR
Node dependency
Native binary
```

All should be scanned.

---

39. Container Digest

Tags can change.

Therefore:

```
latest
```

should not be treated as immutable identity.

Prefer:

```
sha256:<digest>
```

---

40. Kubernetes Scanner

ECDAT should optionally inspect:

```
Deployments
StatefulSets
DaemonSets
Secrets metadata
ConfigMaps
Ingress
Services
Helm
```

The goal is to understand:

```
Which cryptographic assets run where?
```

---

41. Kubernetes Example

```
Deployment
   ↓
Container
   ↓
OpenSSL
   ↓
ECDSA
```

This links infrastructure to cryptographic assets.

---

42. Certificate Scanner

ECDAT should discover certificates from:

```
Repositories
Filesystem
Containers
Load balancers
Ingress
TLS endpoints
PKI systems
```

---

43. X.509 Parsing

Extract:

```
Subject
Issuer
Serial
Validity
SAN
Key Algorithm
Key Size
Signature Algorithm
Key Usage
Extended Key Usage
```

---

44. Certificate Fingerprinting

Use fingerprints to deduplicate certificates:

```
SHA-256 fingerprint
```

This allows the same certificate to be recognized across systems.

---

45. Certificate Relationship

```
Certificate
    ↓
Public Key
    ↓
Algorithm
    ↓
Application
```

```
    ↓
Endpoint
```

This creates valuable graph relationships.

---

46. TLS Scanner

For authorized environments, ECDAT can inspect TLS endpoints.

Potential observations:

```
TLS version
Cipher suite
Certificate
Key exchange
Signature
Server configuration
```

---

47. Network Discovery

The network scanner should be carefully controlled.

It should support:

```
Authorized targets
Port ranges
Rate limits
Timeouts
Credentials
```

Never turn ECDAT into an uncontrolled network scanner.

---

48. Cloud Discovery

ECDAT should eventually support cloud environments.

Potential inventory:

```
Compute
Storage
KMS
Certificates
Load Balancers
API Gateways
```

```
Containers
Functions
```

---

49. Cloud KMS

Discover:

```
Key ID
Algorithm
Key type
Usage
Rotation
Status
Region
Owner
```

Never extract secret key material unnecessarily.

---

50. HSM Discovery

Potential metadata:

```
Vendor
Model
Firmware
Supported Algorithms
Key Inventory Metadata
Partition
Applications
```

Again:

> Metadata, not secret key material.

---

51. CI/CD Scanner

ECDAT should integrate into:

```
GitHub Actions
GitLab CI
Jenkins
Azure DevOps
Other enterprise CI/CD
```

Potential workflow:

```
Build
 ↓
ECDAT Scan
 ↓
CBOM
 ↓
Policy
 ↓
Pass / Warn / Fail
```

---

52. SBOM Integration

If an organization already has:

```
SBOM
```

ECDAT should ingest it.

Why rescan everything if software inventory already exists?

Instead:

```
SBOM
 ↓
Dependency Inventory
 ↓
Crypto Correlation
```

---

53. CBOM Integration

Similarly, existing CBOM data can be imported.

ECDAT should support:

```
Import
Merge
Normalize
Deduplicate
Enrich
```

---

54. Scanner Plugin Architecture

A strong architecture:

```
Scanner Interface
      |
┌─────┬─────┬─────┬─────┐
▼     ▼     ▼     ▼     ▼
Git  Binary Container TLS Cloud
```

Every scanner implements a common interface.

---

55. Scanner Interface

Conceptually:

```
scan(target) -> findings
```

Each finding should contain:

```
scanner
target
timestamp
evidence
confidence
asset_candidate
```

---

56. Scanner Metadata

Every result should record:

```
Scanner Name
Version
Configuration
Timestamp
Target
Toolchain
```

This is important for reproducibility.

---

57. Scanner Versioning

Suppose:

```
Scanner v1:
misses custom crypto

Scanner v2:
detects it
```

ECDAT should know which version produced which finding.

---

58. Scan Orchestration

For a repository:

```
Repository
    │
    ├── Source Scan
    ├── Dependency Scan
    ├── Config Scan
    └── Certificate Scan
```

For a container:

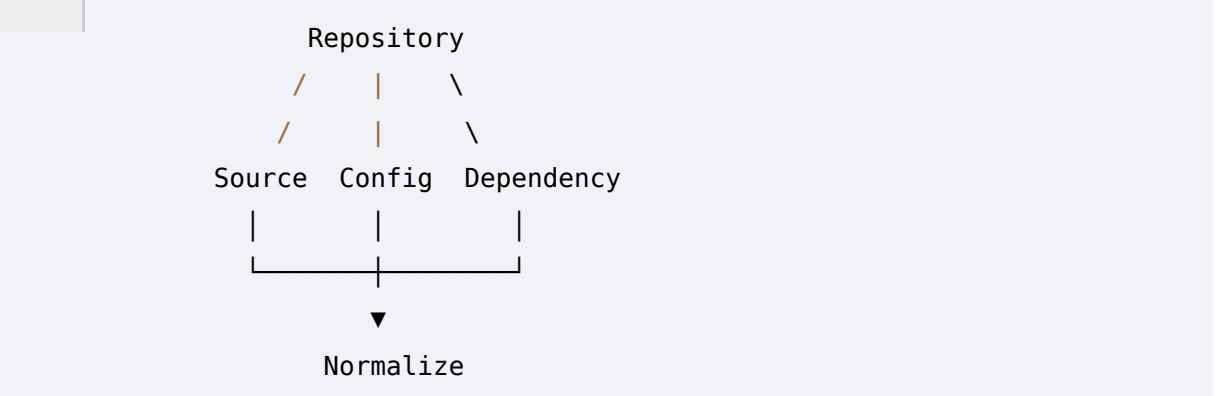
```
Container
    │
    ├── Layer Scan
    ├── Package Scan
    ├── Binary Scan
    └── Config Scan
```

---

59. Parallel Scanning

Independent scans should run in parallel.

Example:

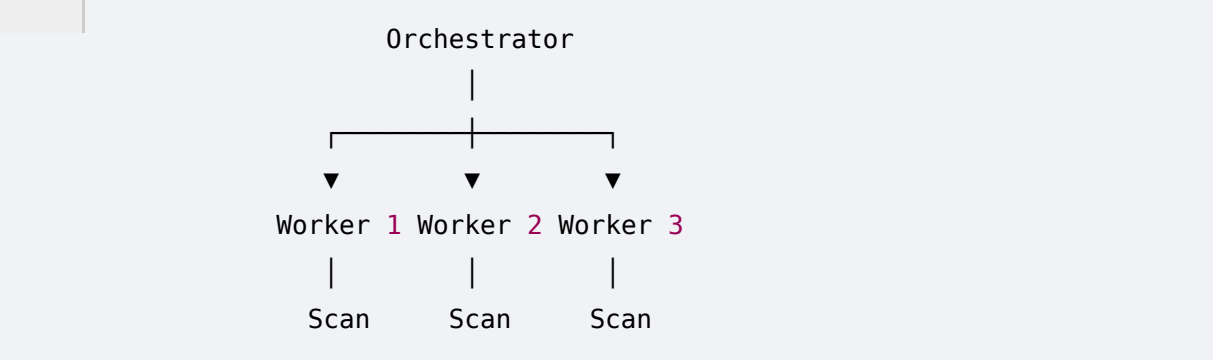


This reduces total scan time.

---

60. Distributed Workers

At enterprise scale:



Workers should be stateless where possible.

---

61. Job Queue

Use a queue:

```
Scan Request
     ↓
Queue
     ↓
Worker
     ↓
Result Queue
```

This allows:

```
Retries
Scaling
Backpressure
Prioritization
```

---

62. Priority Queues

Critical production assets should receive higher priority.

Example:

```
P1 production
   ↓
```

```
P2 production
   ↓
P3 staging
   ↓
P4 development
```

---

63. Retry Logic

Scanners can fail due to:

```
Network
Timeout
Rate Limit
Temporary Service Error
Corrupt Artifact
```

Use:

```
Exponential Backoff
Retry Limits
Failure Classification
```

---

64. Partial Failure

Suppose:

```
Source scan:
SUCCESS

Binary scan:
FAILED

Certificate scan:
SUCCESS
```

The entire scan should not simply become:

```
FAILED
```

Instead:

```
PARTIAL
```

with coverage information.

---

65. Scan Coverage

ECDAT should calculate:

```
Files scanned
Files skipped
Artifacts scanned
Artifacts failed
Endpoints checked
Repositories scanned
```

---

66. Coverage Score

Example:

```
Repository:
10,000 files

Scanned:
9,800

Skipped:
200
```

Coverage:

```
98%
```

This should be visible.

---

67. Why Coverage Matters

A report saying:

> "No cryptographic risk found"

is dangerous if:

```
Only 40% of assets were scanned.
```

ECDAT should instead say:

> "No cryptographic findings were identified within the scanned scope; coverage was 40%."

---

68. Differential Scanning

If only:

```
10 files changed
```

since the previous scan:

```
Scan changed files
+
Affected dependencies
```

rather than the entire repository.

---

69. Dependency-Aware Incremental Scanning

Suppose:

```
crypto/provider.py
```

changes.

ECDAT can identify callers:

```
provider.py
 ↓
auth.py
 ↓
api.py
```

and prioritize rescanning those areas.

---

70. Scan Cache

Cache:

```
File Hash
Artifact Hash
Container Digest
Library Version
```

If unchanged:

```
Reuse Previous Finding
```

This dramatically improves performance.

---

71. Content-Addressable Scanning

Use hashes:

```
SHA-256(file)
```

as identity.

If the same artifact appears in:

```
Repository A
Repository B
Container C
```

ECDAT can recognize it.

---

72. Deduplication

Multiple scanners may report:

```
OpenSSL 3.0.x
```

ECDAT should merge the findings while preserving evidence.

```
Scanner A ─┐
Scanner B ─┼──> Canonical Asset
Scanner C ─┘
```

---

73. Scanner Confidence

Example:

```
Binary symbol:
0.94

Dependency:
0.75
```

```
String:
0.55

AI inference:
0.72
```

Evidence fusion can increase or decrease confidence depending on correlation.

---

74. Finding Normalization

Raw:

```
RSA2048
```

Normalized:

```
algorithm:
RSA

key_size:
2048
```

---

75. Finding Correlation

Example:

```
Source:
ECDSA P-256

Certificate:
ECDSA P-256

TLS:
ECDSA

Runtime:
ECDSA
```

Correlation:

```
Confirmed ECDSA P-256 deployment
```

---

76. Scanner Conflict

Example:

```
Scanner A:
RSA

Scanner B:
ECDSA
```

ECDAT should retain both.

Then:

```
Conflict Resolution
       ↓
Runtime evidence
       ↓
Configuration
       ↓
AI investigation
```

---

77. Source Scanner Output

Example:

```
{
  "finding_id": "f-001",
  "type": "crypto_api",
  "language": "python",
  "file": "security/signing.py",
  "line": 42,
  "api": "sign",
  "candidate_algorithm": "ECDSA",
  "confidence": 0.89
}
```

---

78. Binary Scanner Output

```
{
  "finding_id": "f-002",
  "type": "crypto_symbol",
```

```
  "binary": "/usr/bin/app",
  "symbol": "EVP_PKEY_sign",
  "library": "libcrypto",
  "confidence": 0.91
}
```

---

79. Certificate Scanner Output

```
{
  "finding_id": "f-003",
  "type": "certificate",
  "fingerprint": "sha256:...",
  "algorithm": "ECDSA",
  "curve": "P-256",
  "confidence": 0.99
}
```

---

80. Container Scanner Output

```
{
  "finding_id": "f-004",
  "type": "container_library",
  "image": "registry/app@sha256:...",
  "library": "OpenSSL",
  "version": "3.x",
  "confidence": 0.97
}
```

---

81. Discovery Evidence Pipeline

Everything eventually becomes:

```
Scanner
   ↓
Raw Finding
   ↓
Parser
   ↓
Normalizer
   ↓
```

```
Validator
   ↓
Correlator
   ↓
Evidence Store
   ↓
CBOM
```

---

82. Scan Security

Scanning enterprise environments introduces significant security risks.

Scanner workers should have:

```
Least Privilege
Sandboxing
Network Restrictions
Credential Isolation
Resource Limits
Audit Logging
```

---

83. Repository Security

Do not execute arbitrary repository code merely to scan it.

Prefer:

```
Static Parsing
AST Analysis
Dependency Extraction
Safe Tooling
```

over:

```
Execute unknown build scripts
```

unless explicitly isolated.

---

84. Container Security

Do not automatically run untrusted containers to inspect them.

Prefer:

```
Image Layer Extraction
Filesystem Analysis
Metadata Inspection
Static Binary Analysis
```

---

85. Binary Sandboxing

Binary analysis tools may need isolation.

Use:

```
Sandbox
CPU Limits
Memory Limits
Timeouts
Filesystem Restrictions
```

---

86. Network Scanner Safety

Network discovery must support:

```
Allowlist
Scope
Rate Limit
Timeout
Authentication
Audit
```

This prevents accidental scanning outside authorized infrastructure.

---

87. Credential Architecture

Scanner credentials should be:

```
Short-lived
Scoped
Encrypted
Audited
Rotatable
```

Avoid storing credentials in scanner configuration files.

---

88. Secrets Management

ECDAT should integrate with an enterprise secrets manager where available.

Scanner:

```
Request Credential
 ↓
Secrets Manager
 ↓
Temporary Credential
 ↓
Scan
 ↓
Credential Expired
```

---

89. Air-Gapped Architecture

A strong enterprise deployment could be:

```
┌──────────────────────────────────────┐
│          RESTRICTED NETWORK          │
│                                      │
│   Scanner Workers                    │
│        ↓                             │
│   Evidence Store                     │
│        ↓                             │
│   ECDAT Core                         │
│        ↓                             │
│   Local AI                           │
│                                      │
└──────────────────────────────────────┘
```

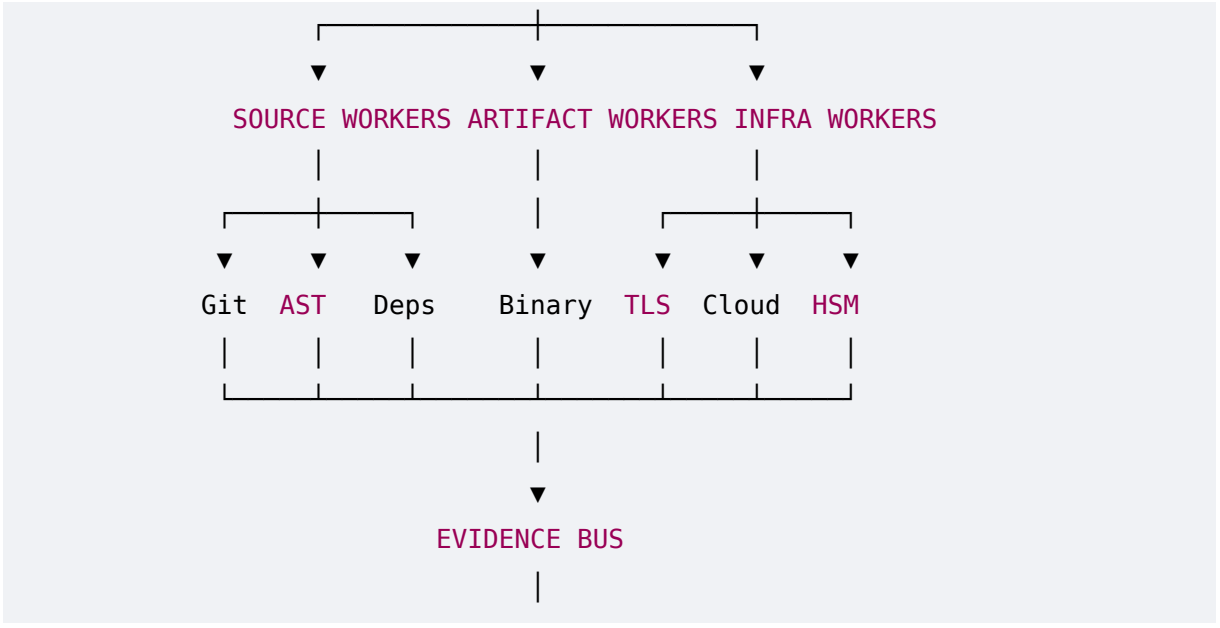
No external connectivity is required.

---

90. Enterprise Architecture

Complete discovery fabric:

```
                    ECDAT CONTROL PLANE
                           |
                    SCAN ORCHESTRATOR
                           |
```



---

91. Scan Scheduling

ECDAT should support policies such as:

```
Critical production:
Daily

Normal production:
Weekly

Staging:
Weekly

Development:
Monthly
```

Actual cadence should be configurable.

---

92. Scan Triggers

Potential triggers:

```
New commit
New release
New container
New certificate
New dependency
Infrastructure change
```

```
Manual request
Scheduled interval
```

---

93. New Dependency Trigger

Example:

```
Developer adds crypto library
        ↓
Dependency event
        ↓
ECDAT scan
        ↓
New asset
        ↓
Risk assessment
```

This provides near-real-time inventory updates.

---

94. Certificate Expiration Trigger

ECDAT can detect:

```
Certificate expiring soon
```

and combine that with:

```
Quantum-vulnerable algorithm
```

to produce:

> Consider migrating the certificate to an approved PQC/hybrid strategy instead of renewing the legacy certificate.

---

95. Scan Health

The platform should expose:

```
Worker Health
Queue Depth
Average Scan Time
Failure Rate
Coverage
```

---

96. Scanner Observability

Each worker should report:

```
CPU
Memory
Duration
Files Processed
Errors
Findings
```

---

97. Scan Performance

Important metrics:

```
Files/sec
MB/sec
Containers/hour
Repositories/hour
Endpoints/minute
```

---

98. Large Repository Handling

A repository may contain:

```
Millions of files
```

Do not load everything into memory.

Use:

```
Streaming
Chunking
Parallel Workers
File Filtering
Incremental Scanning
Caching
```

---

99. Ignored Files

Allow configurable exclusions:

```
node_modules
.git
build
dist
vendor
generated files
```

But exclusions should be visible.

Otherwise coverage becomes misleading.

---

100. Generated Code

Generated code can produce huge numbers of false positives.

ECDAT should classify:

```
Generated
Vendor
Third-party
First-party
Unknown
```

---

101. Third-Party Code

Third-party code should still be inventoried, but ownership matters.

Example:

```
Library:
Vendor Component

Owner:
Vendor

Migration:
Vendor dependency
```

This helps prioritize vendor engagement.

---

102. Source Ownership

ECDAT should correlate:

```
Repository
 ↓
Team
 ↓
Application
 ↓
Business Unit
```

Then risk can be assigned to owners.

---

103. Ownership Example

```
ECDSA
 ↓
Payment API
 ↓
Payments Team
```

Dashboard:

```
Payments Team
Critical Quantum Assets:
17
```

---

104. Discovery Coverage Matrix

ECDAT should show:

Source	Coverage
Git repositories	92%
Containers	88%
Certificates	97%

Cloud	75%
HSM	61%
Network	83%

This is far more meaningful than a single "scan complete" indicator.

---

105. Discovery Confidence

Coverage and confidence are different.

Example:

```
Coverage:
98%

Confidence:
High
```

or:

```
Coverage:
98%

Confidence:
Low
```

if evidence quality is poor.

---

106. Scanner Roadmap

A realistic implementation order:

MVP

```
Source
Dependency
Certificate
Container
```

Version 2

```
Binary
TLS
Kubernetes
SBOM/CBOM
```

Version 3

```
Cloud
HSM
KMS
Runtime
```

This is much more achievable than implementing everything simultaneously.

---

107. What Should Be Demoed at SIH?

For a strong hackathon prototype, demonstrate one complete vertical path:

```
Git Repository
       ↓
Source Scanner
       ↓
Crypto Detection
       ↓
AI Context
       ↓
CBOM
       ↓
Quantum Risk
       ↓
PQC Recommendation
       ↓
Dashboard
```

Then add:

```
Container
Certificate
Binary
```

as secondary scanners.

---

108. Killer Demonstration

Imagine uploading:

```
payment-api/
```

ECDAT scans it.

Output:

```
37 crypto findings
12 algorithms
4 certificates
3 libraries
8 applications/components
```

Then:

```
Critical:
ECDSA P-256
```

ECDAT explains:

```
Found in:
src/auth/sign.py:42

Used for:
Digital signatures

Application:
Payment API

Risk:
Critical

Reason:
Quantum-vulnerable public-key mechanism
protecting long-lived sensitive data.
```

Then:

```
Recommendation:
Hybrid PQC migration
```

This single workflow demonstrates the majority of the problem statement.

---

109. Discovery Engine Requirements

ECDAT should ultimately support:

Source

- Git
- AST
- API detection
- Dependency analysis
- Configuration

Artifacts

- Binary
- Libraries
- Containers
- Packages
- Firmware

Infrastructure

- TLS
- Certificates
- Kubernetes
- Cloud
- KMS
- HSM

Operations

- Full scans
- Incremental scans
- Scheduled scans
- Event scans
- Distributed workers

Security

- Least privilege

- Sandboxing
- Credential isolation
- Air-gapped operation

Quality

- Coverage
- Confidence
- Deduplication
- Provenance
- Retry
- Observability

---

110. The Most Important Architectural Principle

ECDAT should not be:

```
One giant scanner.
```

It should be:

A cryptographic discovery fabric.

Meaning:

```
Many specialized sensors
        ↓
One evidence model
        ↓
One canonical CBOM
```

This makes the platform extensible.

---

111. Why the Plugin Architecture Matters

Tomorrow NTRO may want:

```
New Cloud
New HSM
New Language
New Container Runtime
New Repository
```

```
New Protocol
New Binary Format
```

The core platform should not need to be rewritten.

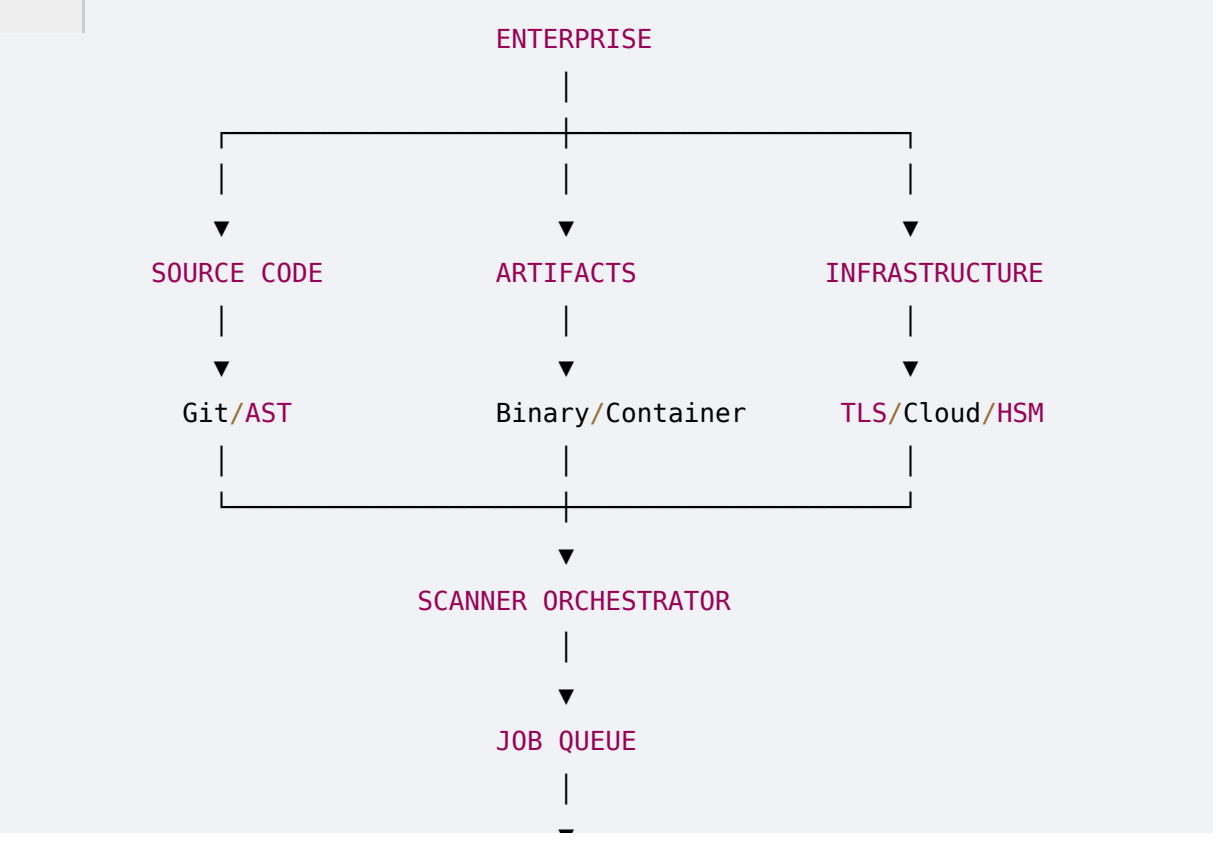
Instead:

```
New Scanner Plugin
        ↓
Common Finding Schema
        ↓
ECDAT
```

---

112. Phase 15 Final Architecture

The complete discovery pipeline:



---

113. Phase 15 Final Takeaway

The most important realization is:

> **Enterprise cryptographic discovery is not a scanning problem. It is a distributed evidence-collection problem.**

A serious system must collect evidence from:

```
Code
Dependencies
Binaries
Containers
Certificates
Protocols
Cloud
HSM
KMS
Runtime
```

and convert all of it into:

```
ONE CANONICAL CRYPTOGRAPHIC INVENTORY.
```

That inventory then feeds:

```
Risk
   ↓
PQC Recommendations
   ↓
Migration
   ↓
Verification
```

PHASE 16 PREVIEW — KNOWLEDGE GRAPH, CRYPTOGRAPHIC DEPENDENCY GRAPH & BLAST-RADIUS ANALYSIS

At this point we have built:

```
Phase 10 → Discovery Architecture
Phase 11 → Canonical CBOM
Phase 12 → Quantum Risk Engine
Phase 13 → PQC Recommendation
Phase 14 → AI Intelligence
Phase 15 → Enterprise Scanner Fabric
```

The next phase is where all of these become **connected**.

Phase 16 will design the:

ECDAT Cryptographic Knowledge Graph

Instead of thinking:

```
RSA = High Risk
```

ECDAT will understand:

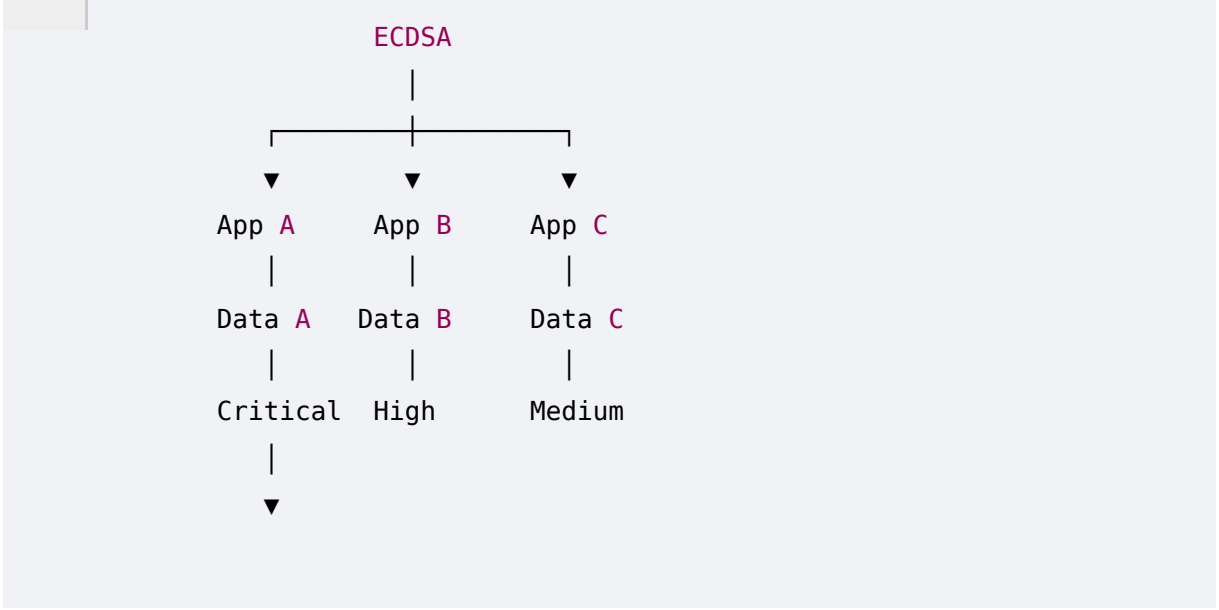
```
RSA
 │
 ├── used by → Application A
 │                  │
 │                  ├── deployed in → Container X
 │                  ├── protects → Sensitive Data
 │                  └── owned by → Team Y
 │
 ├── implemented by → OpenSSL
 │                          │
 │                          └── used by → 2,400 apps
 │
 ├── appears in → Certificate Z
 │
 └── migration target → PQC strategy
```

Phase 16 will cover:

- Graph schema
- Nodes
- Edges
- Cryptographic relationships
- Application dependencies
- Library dependencies
- Certificate chains
- PKI graphs
- Data protection relationships
- Infrastructure relationships
- Business relationships
- Dependency centrality
- Blast radius
- Single points of cryptographic failure

- Shared crypto infrastructure
- Risk propagation
- Migration dependency graphs
- Critical-path discovery
- Attack-path analysis
- Cryptographic concentration
- Algorithm concentration
- Library concentration
- HSM/KMS concentration
- Business impact propagation
- Graph algorithms
- Graph database architecture
- Neo4j-style modelling
- Graph queries
- Visualization
- Interactive investigation
- "Why is this risky?" traversal
- "What breaks if I migrate this?" analysis
- "What depends on this library?" analysis
- "Which applications use RSA?" analysis
- "Which sensitive data is protected by ECDSA?" analysis
- Enterprise blast-radius scoring

The key output will be:



```
            BUSINESS
             PROCESS
```

At that point ECDAT won't merely tell the organization **what cryptography exists**.

It will understand:

what depends on what, what breaks if something changes, and where the enterprise's biggest cryptographic migration bottlenecks actually are.

Phase 15 complete.